

Cheat sheet

Network automation with Ansible validated content

Ansible validated content is a set of collections that contain content such as Ansible plug-ins, roles, and playbooks to address common automation use cases. You can use validated content collections to automate faster and more efficiently with Red Hat Ansible Automation Platform.

The Network Border Gateway Protocol (BGP) validated content collection focuses on platform-agnostic network automation and supports the BGP management experience by providing production-ready content. This cheat sheet covers basic commands and tasks for using Network BGP-validated content:

- The `list` task provides the list of supported resource modules.
- The `gather` task provides the running configuration as facts.
- The `persist` task fetches the running configuration as facts and save these facts as `host_vars` for autogenerated runtime inventory.
- The `deploy` task reads the facts saved as `host_vars` in inventory and deploy them on to the appliance.
- The `health_check` task is used to perform health checks on the specific service.

Get supported resource modules

list

```
ansible.builtin.include_role:  
  name: network.bgp.run  
  vars:  
    actions:  
      - name: list
```

Fetches the supported resource modules for the platform.

Example:

The following playbook gets a list of supported resource modules for the iOS operating system.

```
---
- hosts: iOS
  tasks:
    - name: Get List of supported resource modules
      ansible.builtin.include_role:
        name: network.bgp.run
  vars:
    actions:
      - name: list

$ ansible-playbook -I inventory.yaml list.yaml

TASK [../base/roles/resource_manager : Available Network resources]
*****
ok: [35.155.13.92] => {
  "msg": {
    "ansible_connection": "ansible.netcommon.network_cli",
    "ansible_network_os": "cisco.ios.ios",
    "changed": false,
    "failed": false,
    "modules": [
      "acl_interfaces",
      "acls",
      "bgp_address_family",
      "bgp_global",
      "hostname",
      "interfaces",
      "l2_interfaces",
      "l3_interfaces",
      "lACP",
      "lACP_interfaces",
      "lag_interfaces",
      "lldp_global",
      "lldp_interfaces",
      "logging_global",
      "ntp_global",
      "ospf_interfaces",
      "ospfv2",
      "ospfv3",
      "prefix_lists",
      "route_maps",
      "snmp_server",
      "static_routes",
      "vlans"
    ]
  }
}
```

Create brownfield inventory

persist

```
ansible.builtin.include_role:
  name: network.bgp.run
vars:
  actions:
    - name: persist
```

Fetches and persists the running BGP configuration.

Example:

This task fetches the running config and saves the facts in terms of host_vars on the provided/default inventory path. This inventory can act as a source of truth (SOT) for other operations.

The playbook would look like following:

```
---
- hosts: iOS
  tasks:
    - name: Create run time inventory with host-vars(Facts).
      ansible.builtin.include_role:
        name: network.bgp.run
      vars:
        actions:
          - name: persist

$ ansible-playbook -I inventory.yaml persist.yml
$ tree inventory
inventory
├── host_vars
│   └── 192.168.122.220
│       ├── bgp_address_family.yaml
│       └── bgp_global.yaml
└── inventory.yaml

$ cat inventory/host_vars/192.168.122.220/bgp_global.yaml
bgp_global:
  as_number: '500'
  bgp:
    log_neighbor_changes: true
  neighbors:
    - neighbor_address: 12.0.0.1
      remote_as: '500'
      update_source: Loopback0
    - neighbor_address: 23.0.0.1
      remote_as: '500'
  networks:
    - address: 10.0.0.0
```

Deploy BGP configuration

deploy

```
ansible.builtin.include_role:
  name: network.bgp.run
vars:
  actions:
    - name: deploy
```

Deploys a network BGP configuration.

Example:

This task reads the `host_vars` at runtime from the inventory you created with `persist` and deploys those configuration changes on the network. In the following example, we have added one network:

```
$ vi inventory/host_vars/192.168.122.220/bgp_global.yaml
bgp_global:
  as_number: '500'
  bgp:
    log_neighbor_changes: true
  neighbors:
    - neighbor_address: 12.0.0.1
      remote_as: '500'
      update_source: Loopback0
    - neighbor_address: 23.0.0.1
      remote_as: '500'
  networks:
    - address: 10.0.0.0
    - address: 80.0.0.0

- hosts: iOS
  tasks:
    - name: Deploy configuration
      ansible.builtin.include_role:
        name: network.bgp.run
      vars:
        actions:
          name: deploy

$ ansible-playbook deploy.yml
.
.
.
TASK [network.base.resource_manager : Include tasks]
*****
included: /home/rothakur/ansible-collections/collections/ansible_collections/network/base/roles/
resource_manager/includes/edit_resource.yaml for 192.168.122.220

TASK [network.base.resource_manager : Apply provided configuration]
*****
changed: [192.168.122.220]

TASK [network.base.resource_manager : Apply configuration]
*****
skipping: [192.168.122.220]

PLAY RECAP
*****
192.168.122.220      : ok=16   changed=1    unreachable=0    failed=0    skipped=3    rescued=0
ignored=0
```

Display structured configuration

gather

```
ansible.builtin.include_role:
  name: network.bgp.run
vars:
  actions:
    - name: gather
```

Gathers the running BGP specific configuration.

Example:

This task gathers and displays the running `bgp_global` and `bgp_address_family` configuration from the

```
---
- hosts: iOS
  tasks:
  - name: Gather network bgp configuration
    ansible.builtin.include_role:
      name: network.bgp.run
    vars:
      actions:
        - name: gather

$ ansible-playbook gather.yml
.
.
.
TASK [network.base.resource_manager : Resource Facts]
*****
*****
ok: [192.168.122.220] => {
  "msg": {
    "ansible_connection": "ansible.netcommon.network_cli",
    "ansible_network_os": "cisco.ios.ios",
    "changed": false,
    "failed": false,
    "gathered": {
      "as_number": "500"
    },
    "resource_module_name": "cisco.ios.ios_bgp_address_family"
  },
}

TASK [network.base.resource_manager : Include file write task]
*****
*
skipping: [192.168.122.220]

TASK [network.base.resource_manager : Run the platform facts module]
*****
ok: [192.168.122.220]
```

```
TASK [network.base.resource_manager : Resource Facts]
```

```
*****
*****
ok: [192.168.122.220] => {
  "msg": {
    "ansible_connection": "ansible.netcommon.network_cli",
    "ansible_network_os": "cisco.ios.ios",
    "changed": false,
    "failed": false,
    "gathered": {
      "as_number": "500",
      "bgp": {
        "log_neighbor_changes": true
      },
      "neighbors": [
        {
          "neighbor_address": "12.0.0.1",
          "remote_as": "500",
          "update_source": "Loopback0"
        },
        {
          "neighbor_address": "23.0.0.1",
          "remote_as": "500"
        }
      ],
      "networks": [
        {
          "address": "10.0.0.0"
        },
        {
          "address": "80.0.0.0"
        }
      ]
    },
    "resource_module_name": "cisco.ios.ios_bgp_global"
  }
}
```

Also notice the latest config change, address: 80.0.0.0 that we implemented with deploy.

Perform BGP health checks

health_check

```
ansible.builtin.include_role:
  name: network.bgp.run
vars:
  actions:
    - name: health_checks
      vars:
        checks:
          - name: all_neighbors_up
          - name: all_neighbors_down
          - name: min_neighbors_up
            min_count: 1
```

Fetches the current status of the BGP neighbor for a given network operating system.

Example:

This task enables user to perform certain network BGP health checks as described below:

- `all_neighbors_up` : This health check returns `successful` only when all the BGP neighbors are up and running.
- `all_neighbors_down` : This health check returns `successful` only when all the neighbors are down.
- `min_neighbors_up` : This health check takes `min_count` as input and returns `successful` only when the specified number of neighbors are up and running.

```
---
- hosts: iOS
  gather_facts: false
  tasks:
    - name: Perform BGP Health Checks
      ansible.builtin.include_role:
        name: network.bgp.run
      vars:
        actions:
          - name: health_check
            vars:
              checks:
                - name: all_neighbors_up
                - name: all_neighbors_down
                - name: min_neighbors_up
              min_count: 1

$ ansible-playbook health_check.yml
.
.
.
TASK [network.bgp.run : Set health checks fact]
*****
ok: [192.168.122.220]

TASK [network.bgp.run : BGP health checks]
*****
ok: [192.168.122.220] => {
  "health_checks": {
    "all_neighbors_down": {
      "check_status": "successful",
      "down": 2,
      "total": 2,
      "up": 0
    },
    "all_neighbors_up": {
      "check_status": "failed",
      "down": 2,
      "total": 2,
      "up": 0
    }
  }
}
```

```
"min_neighbors_up": {
  "check_status": "failed",
  "down": 2,
  "total": 2,
  "up": 0
}
}
```

PLAY RECAP

```
*****
192.168.122.220      : ok=5    changed=0    unreachable=0    failed=0    skipped=0    rescued=0
ignored=0
```

You can also use details to get the detailed stats for performed health checks, as shown below:

```
-
- hosts: iOS
  gather_facts: false
  tasks:
    - name: Perform BGP Health Checks
      ansible.builtin.include_role:
        name: network.bgp.run
      vars:
        actions:
          - name: health_check
            vars:
              details: true
              checks:
                - name: all_neighbors_up
                - name: all_neighbors_down
                - name: min_neighbors_up
                  min_count: 1

$ ansible-playbook health_check.yml

ok: [192.168.122.220] => {
  "health_checks": {
    "all_neighbors_down": {
      "check_status": "successful",
      "details": {
        "neighbors": [
          {
            "bgp_table_version": 1,
            "input_queue": 0,
            "msg_rcvd": 0,
            "msg_sent": 0,
            "output_queue": 0,
            "peer": "12.0.0.1",
            "peer_as": 500,
            "peer_state": "Idle",
            "uptime": "13w4d",
            "version": 4
          },
          {
            "bgp_table_version": 1,
            "input_queue": 0,
            "msg_rcvd": 0,
            "msg_sent": 0,
            "output_queue": 0,
            "peer": "23.0.0.1",
            "peer_as": 500,
            "peer_state": "Idle",
            "uptime": "never",
            "version": 4
          }
        ]
      }
    }
  }
}
```



```
    }  
    ]  
    },  
    "down": 2,  
    "total": 2,  
    "up": 0  
  },  
  "all_neighbors_up": {  
    "check_status": "failed",  
    "details": {  
      "neighbors": []  
    },  
    "down": 2,  
    "total": 2,  
    "up": 0  
  },  
  "min_neighbors_up": {  
    "check_status": "failed",  
    "details": {  
      "neighbors": []  
    },  
    "down": 2,  
    "total": 2,  
    "up": 0  
  }  
}  
}
```